JOURNAL OF ALGORITHMS 14, 395–413 (1993)

Adaptive Heapsort*

CHRISTOS LEVCOPOULOS AND OLA PETERSSON

Department of Computer Science, Lund University, Box 118, S-22100 Lund, Sweden

Received November 1990; accepted May 18, 1992

We provide a sorting algorithm, Adaptive Heapsort, that optimally adapts to several known, and new, measures of presortedness. The algorithm is motivated by a new measure, called *Osc*, which intuitively tells the oscillation within the input sequence. We show that Osc generalizes a number of measures appearing in the literature. Moreover, it has an interesting application in the sweepline technique in computational geometry. © 1993 Academic Press, Inc.

1. INTRODUCTION

It is well known that $\Omega(n \log n)$ time is necessary to sort n elements in both the worst case and the average case in a comparison-based model of computation [11]. Despite this fact, some instances seem easier than others and could be sorted faster than indicated by the lower bound. Take, for example, the already sorted sequence or the concatenation of two sorted sequences. Sorting these should not require the same amount of work as sorting a randomly permuted sequence. The fact that some instances of the sorting problem need less resources than a random sequence was first observed by Burge [1] in 1958. Burge identified this *instance easiness* with the amount of existing order (*presortedness*) in the sequence and also proposed measures of existing order.

After being considered by Cook and Kim [3], Dijkstra [4], Knuth [8], and Mehlhorn [11], the concept of presortedness was eventually formalized by Mannila [10] in 1985. Mannila studied the question of how the presortedness of a sequence can be measured. Examples of measures of presortedness that he considered are the number of runs and the number of inversions. Mannila also studied the problem of how a sorting algorithm can take advantage of, and thereby adapt to, the existing order. A sorting

*A preliminary version of this paper was presented at the 1989 Workshop on Algorithms and Data Structures, Ottawa, Canada, August 1989.

0196-6774/93 $5.00
Copyright © 1993 by Academic Press, Inc.
All rights of reproduction in any form reserved.

algorithm is said to be *adaptive* with respect to a measure of presortedness if it sorts all sequences, but performs particularly well on those that have a high degree of presortedness. The more presorted the input sequence is, the faster it should be sorted in the worst case. Moreover, the algorithm should adapt without knowing the amount of presortedness beforehand. Note that most worst-case optimal sorting algorithms, e.g., Heapsort and Mergesort, do not take existing order within their input into account.

In this paper we introduce a new natural measure of presortedness, called *Osc*, originating from a geometric interpretation of the input sequence. Intuitively, Osc measures how much the sequence oscillates. Osc turns out to be a very powerful measure, which generalizes both the number of inversions and the number of runs, in the sense that it captures the existing order measured by both these measures. Another advantage of Osc, compared to most measures appearing in the literature, is that it is invariant with respect to reversing the input sequence. As a consequence it considers a descending sequence to be almost sorted. With its geometric interpretation in mind it is not surprising that Osc has an application in computational geometry, namely in the sweepline technique.

We also provide a sorting algorithm which is optimal with respect to Osc, where optimality means maximum adaptation in an asymptotic sense. The algorithm, *Adaptive Heapsort*, is, as the name indicates, a variant of Heapsort, in the sense that it operates on a heap. The difference from Heapsort is that instead of keeping all the elements in the heap, we store only some of them. If Osc is high, the heap will contain many elements most of the time, and Adaptive Heapsort will run in time $\Theta(n \log n)$. On the other hand, if the oscillation is low, there will only be a few elements in the heap when the operations are performed, and the sorting will be completed considerably faster.

The remainder of the paper is organized as follows. In Section 2 we state some preliminary definitions and introduce the measure Osc. In Section 3 we derive a lower bound for comparison-based sorting algorithms that adapt to Osc. In Section 4 we present Adaptive Heapsort, and in Section 5 we show that it is optimal with respect to Osc as well as to another important measure of presortedness. In Section 6 we compare Osc with some known measures of presortedness. In Section 7 we point out the use of Osc and Adaptive Heapsort in computational geometry. Finally, in Section 8 we summarize and comment on the results obtained.

2. The Oscillation Measure

Let $X = \langle x_1, \ldots, x_n \rangle$ be a *sequence* of n elements x_i from some totally ordered set. For simplicity, we assume that the elements are distinct. For

two sequences, $X = \langle x_1, \ldots, x_n \rangle$ and $Y = \langle y_1, \ldots, y_m \rangle$, their *concatenation* XY is the sequence $\langle x_1, \ldots, x_n, y_1, \ldots, y_m \rangle$. Further, let $|X|$ denote the *length* of X and $\|S\|$ the *cardinality* of a set S. S_n denotes the set of all permutations of $\{1, \ldots, n\}$. A sequence obtained by deleting zero or more elements from X is called a *subsequence* of X. Finally, let $\log x$ denote $\log_2(\max\{2, x\})$.

The new measure of presortedness is motivated by a geometric interpretation of a sequence X. Map each element x_i onto the point (i, x_i) in the plane and draw edges between points that correspond to consecutive elements in X. This gives a polygonal chain corresponding to X. Intuitively, the new measure, *Osc*, tells how much the polygonal chain oscillates. To be more precise, we first define how much one element contributes to the oscillation.

DEFINITION 1. For a sequence X of length n,

$$\text{Cross}(x_i) = \{j \mid 1 \le j < n \text{ and } \min\{x_j, x_{j+1}\} < x_i < \max\{x_j, x_{j+1}\}\},$$

for $1 \le i \le n$.

That is, $\text{Cross}(x_i)$ tells the edges of the polygonal chain that are intersected by a horizontal line through the point (i, x_i).

The new measure of presortedness is just the total number of oscillations, or more formally,

DEFINITION 2. For a sequence X of length n,

$$\text{Osc}(X) = \sum_{i=1}^{n} \|\text{Cross}(x_i)\|.$$

For an illustration of the definitions, see Fig. 1. As well as summing the number of intersections over the points, we can take the sum over the edges. Let us therefore make the following definition.

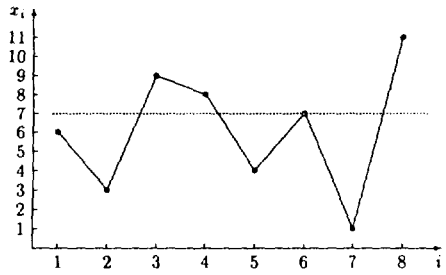


FIG. 1. Illustration of the definitions. Here, for example, $\text{Cross}(x_6) = \{2, 4, 7\}$ and $\text{Osc}(X) = 17$.

DEFINITION 3. For a sequence X of length n,

$$\text{Cover}(x_j, x_{j+1}) = \{i \mid 1 \le i \le n \text{ and } \min\{x_j, x_{j+1}\} < x_i < \max\{x_j, x_{j+1}\}\},$$

for $1 \le j \le n - 1$.

From a geometrical point of view, $\text{Cover}(x_j, x_{j+1})$ contains the (indices of the) points that are located in the horizontal slab spanned by the lines $y = x_j$ and $y = x_{j+1}$.

LEMMA 1. *For any sequence X of length n,*

(a) $\text{Osc}(X) \ge 0$, *with equality if and only if X is ascending or descending.*

(b) $\text{Osc}(\langle x_n, x_{n-1}, \ldots, x_1 \rangle) = \text{Osc}(X)$.

(c) $\text{Osc}(X) = \sum_{i=1}^{n} \|\text{Cross}(x_i)\| = \sum_{j=1}^{n-1} \|\text{Cover}(x_j, x_{j+1})\|$.

(d) $\text{Osc}(X) \le \lfloor n(n-2)/2 \rfloor$, *which is tight.*

Proof. The only claims that may not be obvious are (c) and (d). Let us verify these.

(c) If $j \in \text{Cross}(x_i)$, then $\min\{x_j, x_{j+1}\} < x_i < \max\{x_j, x_{j+1}\}$, by Definition 1. Hence, $i \in \text{Cover}(x_j, x_{j+1})$, by Definition 3. Similarly, if $i \in \text{Cover}(x_j, x_{j+1})$, then $\min\{x_j, x_{j+1}\} < x_i < \max\{x_j, x_{j+1}\}$, and thus $j \in \text{Cross}(x_i)$. We conclude that $j \in \text{Cross}(x_i)$ if and only if $i \in \text{Cover}(x_j, x_{j+1})$.

(d) For simplicity, assume that X is a permutation in S_n. By Definition 1, $\text{Cross}(1) = \text{Cross}(n) = \emptyset$. Furthermore, $\|\text{Cross}(2)\| \le 2$, since the only elements that can contribute to Cross(2) are one and the element that precedes one in X. Similarly, $\|\text{Cross}(n-1)\| \le 2$. In general, we have $\|\text{Cross}(i)\| \le 2(i-1)$, for $i \le \lfloor n/2 \rfloor$, since the elements contributing must either be smaller than i or precede an element which is smaller than i. Similarly, for $i \ge \lfloor n/2 \rfloor$, we have $\|\text{Cross}(i)\| \le 2(i-1)$. For even n's we therefore have

$$\text{Osc}(X) = \sum_{i=1}^{n} \|\text{Cross}(x_i)\| \le 2 \sum_{i=1}^{n/2} 2(i-1) = \frac{n(n-2)}{2}. \qquad (1)$$

For odd n's, $\|\text{Cross}(\lceil n/2 \rceil)\| \le n - 2$, because $\lceil n/2 \rceil$ itself cannot contribute and neither can the element that precedes $\lceil n/2 \rceil$ in X. (If there is

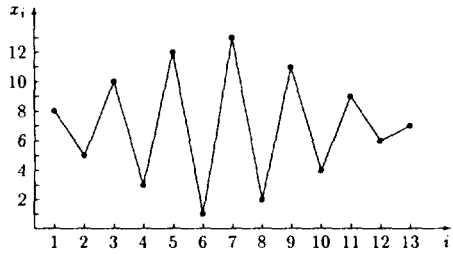


FIG. 2. The upper bound in Lemma 1(d) is tight for the sequence $X = \langle 8, 5, 10, 3, 12, 1, 13, 2, 11, 4, 9, 6, 7 \rangle$.

no element that precedes $\lceil n/2 \rceil$, then $\lceil n/2 \rceil$ is the first element in X, in which case the last element in X cannot contribute to Cross($\lceil n/2 \rceil$).) If n is odd, we thus have

$$\mathrm{Osc}(X) = \sum_{i=1}^{n} \|\mathrm{Cross}(x_i)\| = \|\mathrm{Cross}(\lceil n/2 \rceil)\| + \sum_{x_i \neq \lceil n/2 \rceil} \|\mathrm{Cross}(x_i)\|$$

$$\leq n - 2 + 2 \sum_{i=1}^{(n-1)/2} 2(i-1) = \frac{n(n-2) - 1}{2},$$

which together with Eq. (1) proves the upper bound.

Figure 2 shows a small example sequence, which can easily be extended to arbitrary large n, that attains the proved worst-case bound. Essentially, the sequence is constructed as follows. Start with a sorted sequence X. Let X_i, $1 \leq i \leq 2$, be the subsequence of the first half of X consisting of every second element, starting with the ith. Similarly, decompose the second half of X into X_3 and X_4. Next, let $X_{1,2}$ denote the concatenation of the reverse of X_1 with X_2. Similarly, let $X_{3,4}$ denote the concatenation of X_3 and the reverse of X_4. Finally, make a perfect shuffle of $X_{1,2}$ and $X_{3,4}$. □

3. OPTIMALITY

The concept of an optimal algorithm with respect to a measure of presortedness was given in a general form by Mannila [10]. We use the following equivalent definitions [2, 9, 13].

DEFINITION 4. Let M be a measure of presortedness, and let T_n be the set of comparison trees for the set S_n. Then, for any $k \geq 0$ and $n \geq 1$,

$$C_M(n,k) = \min_{T \in T_n} \max_{\pi \in \text{below}_M(n,k)} \{\text{number of comparisons spent by } T \text{ to sort } \pi\},$$

where $\text{below}_M(n,k) = \{\pi | \pi \in S_n \text{ and } M(\pi) \leq k\}$.

DEFINITION 5. Let M be a measure of presortedness, and let S be a comparison-based sorting algorithm that uses $T_S(X)$ steps on input X. We say that S is *M-optimal*, or *optimal with respect to* M, if $T_S(X) = O(C_M(|X|, M(X)))$.

When proving optimality of an adaptive sorting algorithm the following theorem is helpful [2, 9, 13].

THEOREM 2. *Let M be a measure of presortedness. Then*

$$C_M(n,k) = \Theta(n + \log\|\text{below}_M(n,k)\|).$$

A lower bound for comparison-based sorting algorithms with respect to a measure of presortedness is thus obtained by bounding the cardinality of the below-set from below.

We next derive a lower bound on $C_{\text{Osc}}(n,k)$. First, we state a general lemma that can be used to obtain new lower bounds from known ones [12].

LEMMA 3. *Let M_1 and M_2 be measures of presortedness. If there exists a function f such that $M_1(X) \leq f(n) \cdot M_2(X)$, for any sequence X of length n, then $C_{M_1}(n,k) \geq C_{M_2}(n, k/f(n))$.*

Proof. It is easy to see that $\text{below}_{M_1}(n,k) \supseteq \text{below}_{M_2}(n, k/f(n))$, from which the lemma follows by Definition 4. □

The lower bound we will take advantage from in order to derive one on $C_{\text{Osc}}(n,k)$ is the bound on $C_{\text{Runs}}(n,k)$. A *run* is an ascending consecutive subsequence of maximal length and the measure *Runs* tells the number of runs in a sequence [8, 10]. More formally, for a sequence X of length n,

$$\text{Runs}(X) = \|\{i | 1 \leq i < n \text{ and } x_i > x_{i+1}\}\| + 1.$$

Next, we need an upper bound on Osc in terms of Runs.

LEMMA 4. *For any sequence X of length n, $\text{Osc}(X) \leq 2n \cdot \text{Runs}(X)$.*

Proof. Not counting the last elements in the runs in X, at most one element in each run can contribute to $\text{Cross}(x_i)$, $1 \leq i \leq n$, because the runs are ascending. Furthermore, since the number of elements being last

in some run is Runs(X), it follows that

$$\text{Osc}(X) = \sum_{i=1}^{n} \|\text{Cross}(x_i)\| \leq \sum_{i=1}^{n} 2 \cdot \text{Runs}(X) = 2n \cdot \text{Runs}(X),$$

which completes the proof. □

Combining this lemma with the fact that $C_{\text{Runs}}(n, k) = \Omega(n \log k)$, as proved by Mannila [10], and Lemma 3 gives

LEMMA 5. $C_{\text{Osc}}(n, k) = \Omega(n \log(k/n))$.

4. The Algorithm

Adaptive Heapsort is a variant of Heapsort. Recall that Heapsort starts by building a heap consisting of n elements and then repeatedly performs the operation *ExtractMax*, n times. The number of elements in the heap during most *ExtractMax* operations is $\Theta(n)$. Since the time taken by each *ExtractMax* is logarithmic in the size of the heap, the total running time of Heapsort is $\Theta(n \log n)$. Adaptive Heapsort operates on a heap too. Instead of storing *all* elements in the heap, however, only the ones that can possibly be the maximum of the remaining elements, the *max-candidates*, are stored. Initially, we spend linear preprocessing time to construct a data structure that supports efficient retrieval of new max-candidates, to be inserted into the heap before each *ExtractMax* is performed. This data structure, which is described below, is the Cartesian tree. We proceed with a high-level description of Adaptive Heapsort.

PROCEDURE *Adaptive Heapsort* (X: *sequence*).

```
    Construct the Cartesian tree 𝒞(X)
    Insert the root of 𝒞(X) in a heap
    for i := 1 to n do
        Perform ExtractMax on the heap
        if the extracted element has any children in 𝒞(X) then
            Retrieve the children from 𝒞(X)
            Insert the children in the heap
        endif
    endfor
end
```

4.1. *The Cartesian Tree*

The *Cartesian tree* for a sequence $X = \langle x_1, \ldots, x_n \rangle$, $\mathscr{C}(X)$, was introduced by Vuillemin [15] as the binary tree with root $x_i = \max\{x_1, \ldots, x_n\}$.

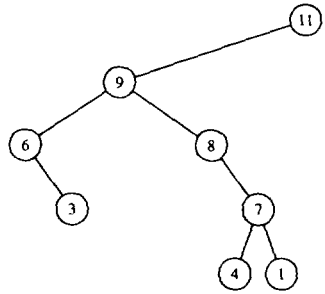


FIG. 3. The Cartesian tree for $\langle 6, 3, 9, 8, 4, 7, 1, 11 \rangle$.

Its left subtree is the Cartesian tree for $\langle x_1, \ldots, x_{i-1} \rangle$ and its right subtree is the Cartesian tree for $\langle x_{i+1}, \ldots, x_n \rangle$. The Cartesian tree for the sequence of length zero is the empty binary tree (see Fig. 3).

An algorithm for computing the Cartesian tree in linear time was given by Gabow, Bentley, and Tarjan [6]. Their algorithm constructs a sequence of Cartesian trees $\mathscr{C}_1, \mathscr{C}_2, \ldots, \mathscr{C}_n$, where $\mathscr{C}_i$ is the Cartesian tree for the subsequence $\langle x_1, x_2, \ldots, x_i \rangle$. During the construction we keep a pointer to the element last inserted. $\mathscr{C}_i$ is derived by inserting x_i into $\mathscr{C}_{i-1}$ as follows. Start at x_{i-1} and ascend along the rightmost path of $\mathscr{C}_{i-1}$ until the first element x_k which is greater than x_i is found. The right subtree of x_k is made the left subtree of x_i, and x_i is made the right child of x_k. If no element greater than x_i is found, the root of $\mathscr{C}_{i-1}$ is reached, in which case x_i is made the new root having $\mathscr{C}_{i-1}$ as its left subtree. As a node leaves the rightmost path immediately after it has been traversed, it is not hard to see that the given algorithm runs in linear time. We will, however, prove the following tight bound.

LEMMA 6. *The above algorithm computes the Cartesian tree for a sequence of n elements in at most* $2n - 3$ *comparisons, which is tight.*

Proof. Let c_i denote the number of element comparisons performed when inserting the element x_i into $\mathscr{C}_{i-1}$. Clearly, $c_1 = 0$. Further, let l_i denote the number of nodes in the rightmost path in $\mathscr{C}_i$. Then $l_1 = 1$. For $i \geq 2$, we have

$$c_i \leq l_{i-1} - l_i + 2, \tag{2}$$

since if we make c_i comparisons when inserting x_i, at least $c_i - 1$ nodes are removed from the rightmost path of $\mathscr{C}_{i-1}$, and the node x_i is added. Hence, the decrease in length of the rightmost path is at least $c_i - 2$, from which Eq. (2) follows.

The total number of comparisons performed is

$$\sum_{i=1}^{n} c_i = \sum_{i=2}^{n} c_i \leq \sum_{i=2}^{n} (l_{i-1} - l_i + 2) = 2(n-1) + l_1 - l_n. \qquad (3)$$

We might as well assume that x_n is not the greatest element in X. (If it is, just interchange x_n and the second greatest element, which does not change c_i, $1 \leq i \leq n$.) Then $l_n \geq 2$, and as $l_1 = 1$, Eq. (3) gives that the total number of comparisons spent is bounded by $2n - 3$.

The sequence $\langle n, n-2, n-3, \ldots, 2, 1, n-1 \rangle$ shows that the bound is tight in the worst case. □

4.2. *Putting Things Together*

As already mentioned, the Cartesian tree $\mathscr{C}(X)$ is used in Adaptive Heapsort to support efficient retrieval of the new max-candidates. Initially, the only max-candidate is the root of $\mathscr{C}(X)$, which is inserted into the empty heap. In each of the following iterations, the new max-candidates are the children in $\mathscr{C}(X)$ of the element last extracted from the heap.

The correctness of Adaptive Heapsort can straightforwardly be proved by contradiction. It is enough to prove that the elements are extracted from the heap in descending order. Assume that when *ExtractMax* returns the element x_i, then there exists some element in the Cartesian tree which is greater than x_i, and which has not yet been extracted from the heap. (No such element can be in the heap by the definition of *ExtractMax*.) Let x_k be the greatest such element in the Cartesian tree. Since the parent of x_k in $\mathscr{C}(X)$ is greater than x_k it has already been extracted from the heap. But as an element is inserted into the heap once its parent is extracted, x_k must be present in the heap, contradicting our assumption.

4.3. *Implementation Considerations*

When building the Cartesian tree we need pointers from children to parents, since we traverse it in that direction. For convenience, we also keep pointers from parents to children. Once the tree has been constructed, the pointers pointing upward are no longer required and can be used for implementing the heap. During the sorting phase, a node in $\mathscr{C}(X)$ thus contains the corresponding array element and two children pointers. The heap is just an array of pointers to the nodes in the Cartesian tree. This guarantees that given the element last extracted from the heap, its children in $\mathscr{C}(X)$ can be found in constant time. We

conclude that the amount of space needed in Adaptive Heapsort is $3n$ pointers in addition to the input elements.

5. Upper Bounds

In order to prove that the time consumed by Adaptive Heapsort matches Lemma 5 we need the following two lemmas.

Lemma 7. *For each i, when x_i is extracted from the heap, the heap contains at most $\lfloor \|\text{Cross}(x_i)\|/2 \rfloor + 2$ elements.*

Proof. From a geometrical point of view a horizontal line through the point (i, x_i) divides the polygonal chain that corresponds to X into a number of subchains. In each subchain either all points are below the line or all points are above the line. Denote by a *valley* a subchain in which all points are below the line. Each valley, possibly except the leftmost and the rightmost, is defined by two intersections with the polygonal chain. The number of valleys is therefore at most $\lfloor \|\text{Cross}(x_i)\|/2 \rfloor + 2$. When x_i is extracted from the heap, all other elements in the heap are smaller than x_i and must therefore come from the valleys. The lemma follows if we can prove that each valley contributes at most one element to the heap at the moment x_i is extracted.

Suppose x_{j_1} and x_{j_2}, $j_1 < j_2$, are two elements in the heap which originate from the same valley. Let x_{j_0} be the lowest common ancestor of x_{j_1} and x_{j_2} in $\mathscr{C}(X)$. Since an element is inserted into the heap only when its parent in $\mathscr{C}(X)$ is extracted from the heap, x_{j_0} must have been extracted earlier, and hence $x_{j_0} > x_i$. Using the same argument implies that $j_0 \neq j_1$ and $j_0 \neq j_2$. The definition of $\mathscr{C}(X)$ guarantees, however, that $j_1 < j_0 < j_2$. Therefore, x_{j_1} and x_{j_2} cannot belong to the same valley, which contradicts the assumption and concludes the proof. □

Lemma 8. *Adaptive Heapsort sorts a sequence X of length n in time*

$$O\left(n + \sum_{i=1}^{n} \log\|\text{Cross}(x_i)\|\right).$$

Proof. In each iteration we perform at most two operations on $\mathscr{C}(X)$; two retrievals, and three on the heap; one *ExtractMax* and two insertions. Given the element last extracted from the heap, the retrievals from $\mathscr{C}(X)$ can be done in constant time, as discussed in Section 4.3. The complexity of the heap operations are logarithmic in the number of elements in the heap, which is not greater than $\lfloor \|\text{Cross}(x_i)\|/2 \rfloor + 2$, by Lemma 7. Since the construction of the Cartesian tree was done in linear time, by Lemma

6, and as the total time for all retrievals from the tree is linear, the time complexity of Adaptive Heapsort is

$$O\left(n + \sum_{i=1}^{n} \log\|\text{Cross}(x_i)\|\right),$$

which completes the proof. □

THEOREM 9. *Adaptive Heapsort sorts a sequence X of length n with* $\text{Osc}(X) = k$ *in time* $O(n \log(k/n))$, *which is optimal with respect to Osc.*

Proof. We show that the sum in Lemma 8 is bounded by $O(n \log(k/n))$:

$$\sum_{i=1}^{n} \log\|\text{Cross}(x_i)\| = \log\left(\prod_{i=1}^{n} \|\text{Cross}(x_i)\|\right)$$

$$= n \log\left(\prod_{i=1}^{n} \|\text{Cross}(x_i)\|^{1/n}\right).$$

We next use the fact that the geometric mean can never be larger than the arithmetic mean. That is, for any positive numbers r_i, $1 \le i \le n$, it holds that $\prod_{i=1}^{n} r_i^{1/n} \le (\sum_{i=1}^{n} r_i)/n$. Therefore,

$$n \log\left(\prod_{i=1}^{n} \|\text{Cross}(x_i)\|^{1/n}\right) \le n \log\left(\frac{1}{n} \cdot \sum_{i=1}^{n} \|\text{Cross}(x_i)\|\right)$$

$$= O\left(n \log\left(\frac{\text{Osc}(X)}{n}\right)\right).$$

Hence, the total time spent by Adaptive Heapsort is $O(n \log(k/n))$, which is optimal with respect to *Osc*, by Lemma 5. □

Since Adaptive Heapsort is more or less tailored for Osc, it may seem that Theorem 9 precisely expresses its adaptivity. It does, however, not tell the whole truth. We will next show that if we are careful when implementing the operations on the heap then the algorithm is optimal with respect to another significant measure of presortedness, namely *Block* [2]. In a sequence, let a *block* be a consecutive subsequence that also appears in the sorted sequence. Then $\text{Block}(X)$ tells the number of blocks in a sequence X and is formally defined as

$$\text{Block}(X) = \|\{1 \le i < |X| \mid x_i \text{ gets a new right neighbor during the sort}\}\| + 1.$$

Adaptivity with respect to Block is important in that it implies adaptivity with respect to two other widely known measures as well (see Section 6).

To analyze the performance of Adaptive Heapsort in terms of Block we make use of the following lemma

LEMMA 10. *Let H be a heap with $\|H\|$ elements taken from a sequence X, and let x_i be the maximum element in H. Then, extraction of x_i followed by insertion of x_j in H can be performed in two comparisons if $j = i - 1$ and if x_i and x_{i-1} belong to the same block in X; otherwise it takes at most $2 \cdot \log\|H\|$ comparisons.*

Proof (We assume some familiarity with the definition of a heap [8]). Consider an extraction of the maximum element, followed by an insertion. Rather than rearranging the heap after the root extraction and inserting the new element in a bottom–up fashion, we make the insertion top–down, starting from the empty root and sifting it down to the correct position, performing two comparisons per level.

Let x_i and $x_j = x_{i-1}$ belong to the same block. Then, by the definition of a block, when x_i has been extracted from H, x_{i-1} is greater than all the elements present in H. Consequently, when x_{i-1} is inserted it will remain in the root of H, and thus, the extraction and insertion take only two comparisons. If x_i is not the successor of x_j in X then x_j might be sifted down to a leaf, which takes $2 \cdot \log\|H\|$ comparisons. □

By the definition of the Cartesian tree, each block in the input sequence gives rise to a left chain in the tree, in which all nodes, possibly except those corresponding to the first and last elements in the block, have no right children. When inserting two elements in the heap, we always insert the one corresponding to the left child first, which ensures that it will take constant time in case it belongs to the same block as the element that was extracted. Hence, by Lemma 10, at most two elements (the first and the last) per block can take more than constant time to process during the sort. Moreover, two elements from the same block cannot be in the heap simultaneously, because the smaller of them is not inserted in the heap until the greater one has been extracted, by the definition of the Cartesian tree. Therefore, the cardinality of the heap cannot exceed the number of blocks in the input sequence. Thus, Adaptive Heapsort runs in $O(n + \text{Block}(X)\log \text{Block}(X))$ time. Since this bound matches the lower bound given by Carlsson, Levcopoulos, and Petersson [2], we have proved

THEOREM 11. *Adaptive Heapsort sorts a sequence X of length n with $\text{Block}(X) = b$ in time $O(n + b \log b)$, which is optimal with respect to Block.*

It is of interest to compare the exact worst-case bound on the number of comparisons of Adaptive Heapsort to that of (ordinary) Heapsort. It is clear that n insertions and n extractions are performed on the heap. Note that, since the Cartesian tree is binary, at least $n/2$ of the insertions are immediately preceded by an extraction. These extractions together with their following insertions take at most $n \log n$ comparisons, by Lemma 10. The remaining at most $n/2$ extractions may take $2 \log n$ comparisons each, which sums up to another $n \log n$ term. Finally, the remaining, at most $n/2$, insertions take at most $\log n$ comparisons each, because one comparison per level suffices when they are inserted in a bottom–up fashion. Hence, a straightforward implementation of Adaptive Heapsort runs in at most $2.5n \log n$ comparisons. As the usual implementation of Heapsort uses $2n \log n$ comparisons we conclude that Adaptive Heapsort uses only 25% more comparisons in the worst case. Note, however, that Heapsort is an in-place algorithm, while Adaptive Heapsort uses $3n$ extra pointers.

6. A Comparison with Other Measures

We show that Osc is a generalization of both the number of runs and the number of inversions, in the sense that it captures sequences that are presorted according to any of these measures. More precisely, we show that any Osc-optimal sorting algorithm is also optimal with respect to these measures. To perform the comparison we make use of the framework introduced by Petersson and Moffat [13]. They compared measures based on the relation between the values of $C_M(n, k)$, as follows.

Lemma 12. *Let M_1 and M_2 be measures of presortedness. If*

$$C_{M_1}(|X|, M_1(X)) = O\big(C_{M_2}(|X|, M_2(X))\big),$$

then every M_1-optimal sorting algorithm is M_2-optimal.

Proof. Immediate from Definition 5. □

To use this lemma, we need a method to derive upper bounds on $C_M(n, k)$, which can be obtained from upper bounds on sorting algorithms [2, 13].

Lemma 13. *Let M be a measure of presortedness, for which $C_M(n, k) = \Omega(f(n, k))$, where f is non-decreasing on n and k. If there exists a comparison-based sorting algorithm that sorts a sequence X in $O(f(|X|, M(X)))$ comparisons then $C_M(n, k) = \Theta(f(n, k))$.*

By Theorem 9, the upper bound on Adaptive Heapsort matches the lower bound in Lemma 5, and thus

$$C_{\mathrm{Osc}}(n, k) = \Theta(n \log(k/n)),$$

by Lemma 13. Applying Lemma 4 now gives

$$\begin{aligned} C_{\mathrm{Osc}}(|X|, \mathrm{Osc}(X)) &= \Theta\left(|X| \log\left(\frac{\mathrm{Osc}(X)}{|X|}\right)\right) \\ &= O(|X| \log \mathrm{Runs}(X)) \\ &= O(C_{\mathrm{Runs}}(|X|, \mathrm{Runs}(X))), \end{aligned}$$

where the last equality follows from Mannila's lower bound [10]. Hence, by Lemma 12,

LEMMA 14. *Every Osc-optimal sorting algorithm is Runs-optimal.*

To see that the lemma is not true the other way around, consider the sequence $X = \langle 2, 1, 4, 3, \ldots, n, n-1 \rangle$. We have that $\mathrm{Osc}(X) = n/2$ and $\mathrm{Runs}(X) = n/2$. Hence, every Osc-optimal sorting algorithm sorts X in linear time while a Runs-optimal one is allowed to spend $\Theta(n \log n)$ time.

To show that Osc is also superior to the number of inversions, let us first make a formal definition. For a sequence X of length n,

$$\mathrm{Inv}(X) = \left\| \{(i, j) | 1 \le i < j \le n \text{ and } x_i > x_j\} \right\|.$$

Guibas *et al.* [7] proved

LEMMA 15. $C_{\mathrm{Inv}}(n, k) = \Theta(n \log(k/n))$.

As for Runs, we next need an upper bound on Osc in terms of Inv.

LEMMA 16. *For any sequence* X, $\mathrm{Osc}(X) \le 4 \cdot \mathrm{Inv}(X)$.

Proof. By Lemma 1(c) and Definition 3,

$$\begin{aligned} \mathrm{Osc}(X) &= \sum_{j=1}^{n-1} \left\| \mathrm{Cover}(x_j, x_{j+1}) \right\| \\ &= \sum_{j=1}^{n-1} \left\| \{i | 1 \le i \le n \text{ and } \min\{x_j, x_{j+1}\} < x_i < \max\{x_j, x_{j+1}\}\} \right\|. \end{aligned}$$

To bound this sum from above in terms of $\mathrm{Inv}(X)$, recall the geometric interpretation of each term, as mentioned in Section 2. To the left of each edge we extend the horizontal slab upwards, and to the right we extend it

downwards. That is,

$$\mathrm{Osc}(X) \le \sum_{j=1}^{n-1} \left\| \{ i \mid 1 \le i < j \text{ and } x_i > \min\{x_j, x_{j+1}\} \} \right\| \tag{4}$$

$$+ \sum_{j=1}^{n-1} \left\| \{ i \mid j < i \le n \text{ and } x_i < \max\{x_j, x_{j+1}\} \} \right\|. \tag{5}$$

Suppose $k < l$ and $x_k > \min\{x_l, x_{l+1}\}$, that is, k contributes to the lth component of the sum in (4). Without loss of generality, assume that $x_l < x_{l+1}$. Then (k, l) is an inversion in X by the definition of Inv. The only other component of the sum to which this inversion can contribute to is the $(l-1)$th. Hence, each inversion in X contributes at most two to the sum in (4).

A similar argument gives that each inversion in X can contribute at most two to the sum in (5). Each one of the two sums is thus bounded by $2 \cdot \mathrm{Inv}(X)$, and the lemma follows. □

Let $n = 2m$ for some integer m. Then, for the sequence

$$X = \left\langle 1, \frac{n}{2} + 1, 2, \frac{n}{2} + 2, \ldots, \frac{n}{2}, n \right\rangle$$

it holds that $\mathrm{Osc}(X) = 4 \cdot \mathrm{Inv}(X) - n + 2$, proving that the bound in Lemma 16 is tight up to an additive linear term.

LEMMA 17. *Every Osc-optimal sorting algorithm is Inv-optimal.*

Proof. By the tight bound on $C_{\mathrm{Osc}}(n, k)$ and Lemma 16 we have

$$C_{\mathrm{Osc}}(|X|, \mathrm{Osc}(X)) = \Theta\left(|X| \log\left(\frac{\mathrm{Osc}(X)}{|X|} \right) \right) = O\left(|X| \log\left(\frac{\mathrm{Inv}(X)}{|X|} \right) \right),$$

which is $O(C_{\mathrm{Inv}}(|X|, \mathrm{Inv}(X)))$, by Lemma 15. Again, the claim follows by Lemma 12. □

Since a descending sequence is sorted in linear time by every Osc-optimal sorting algorithm, while an Inv-optimal one is allowed to use $\Theta(n \log n)$ time, it follows that the lemma is not true the other way around.

Let us also consider the measure *Max*, defined as the maximum distance an element is from its correct position [5].

LEMMA 18. *Every Inv-optimal sorting algorithm is Max-optimal.*

Proof. Using the facts that $\mathrm{Inv}(X) \leq n \cdot \mathrm{Max}$ and $C_{\mathrm{Max}}(n, k) = \Omega(n \log k)$, as proved by Estivill-Castro and Wood [5], the proof is analogous to those of the preceding lemmas. □

To summarize the above observations, we have

THEOREM 19. *Every Osc-optimal sorting algorithm is optimal with respect to the measures Inv, Max, and Runs.*

To appreciate the adaptivity of Adaptive Heapsort in full we consider two other commonly used measures of presortedness, namely *Rem* and *Exc* [3, 10]. Rem tells the minimum number of elements that have to be removed from a sequence in order to leave a sorted sequence; Exc tells the minimum number of exchanges of arbitrary elements needed to bring a sequence into sorted order. Carlsson, Levcopoulos, and Petersson [2] proved that every Block-optimal sorting algorithm is Rem-optimal, and similarly, that every Rem-optimal sorting algorithm is Exc-optimal. (The results are not true the other way around.) It is easy to find a sequence showing that *not* every Osc-optimal sorting algorithm is Exc-optimal, and hence not optimal with respect to Rem or Block by transitivity [12].

Combining the preceding results shows that Adaptive Heapsort is optimally adaptive with respect to most known measures of presortedness:

COROLLARY 20. *Adaptive Heapsort is optimal with respect to the measures Osc, Inv, Max, Block, Rem, Exc, and Runs.*

7. AN APPLICATION

Adaptive Heapsort has an interesting application in the *sweepline technique* in computational geometry [14]. The sweepline technique is, for example, often used when decomposing polygons into subpolygons which satisfy certain conditions, such as being triangular or monotone. Let $P = \langle v_1, v_2, \ldots, v_n \rangle$ be a polygon where the vertices $v_i = (x_i, y_i)$ are given in clockwise (or counterclockwise) order. Sweepline algorithms comprise a preprocessing step in which the vertices of the polygon are sorted according to one of its coordinates, say the y-coordinate. Then a horizontal line is swept over the polygon, starting at the last vertex (according to the sorted order). During the sweep, information relevant for solving the problem is updated at each new vertex encountered. The time complexity of the update depends on the amount of data in the data structure keeping the information. This amount, in turn, is usually proportional to the number of polygonal edges currently crossed by the sweepline. If the

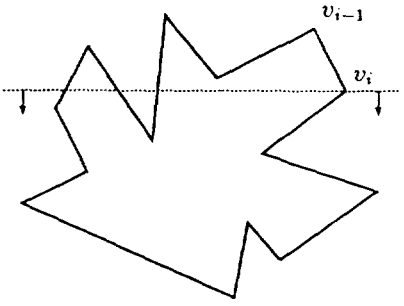


FIG. 4. The sweepline has reached the vertex $v_i = (x_i, y_i)$. The number of polygonal edges intersected by the sweepline equals $\|\text{Cross}(y_i)\|$, where $Y = \langle y_1, y_2, \ldots, y_n \rangle$.

data structure supports logarithmic time updating, the time consumed by the sweep will be the same as that of Adaptive Heapsort when applied to sort the coordinates, by Lemma 8 (see Fig. 4). Hence, Adaptive Heapsort is a good candidate in the preprocessing step for sweepline algorithms, since it takes the same amount of time as does the sweep. This might be considerably less than $\Theta(n \log n)$, which is always needed if, for example, (ordinary) Heapsort is used.

It should be noted that the oscillation highly depends on the orientation of the polygon in the plane; a slight rotation might increase the oscillation from $\Theta(n)$ to $\Theta(n^2)$. An interesting question is, therefore, whether there is a fast algorithm for determining an orientation for which the oscillation approximates the minimum.

8. Conclusions

We introduced a new natural measure of presortedness, called Osc, which measures how much the input sequence oscillates when interpreted as a polygonal chain in the plane. Osc was proved to be superior both to the number of inversions and to the number of runs.

We further presented the sorting algorithm Adaptive Heapsort, which is optimal with respect to Osc and Block. As opposed to Heapsort, Adaptive Heapsort adapts to the oscillation within the input sequence by keeping just the necessary elements in the heap, instead of all of them. The algorithm is simple and fairly space efficient. For instance, after a linear time preprocessing phase, the only data structures used are a heap and a static tree. Moreover, Adaptive Heapsort has an application in the preprocessing step of sweepline algorithms in computational geometry.

Finally, let us comment on the problem of computing $\mathrm{Osc}(X)$. It is not hard to see that $\mathrm{Osc}(X)$ can be computed in linear time once X has been sorted. The key observation is that

$$\|\mathrm{Cover}(x_j, x_{j+1})\| = |\pi(j) - \pi(j+1)|,$$

where $\pi(j) = \|\{i \mid 1 \le i \le n \text{ and } x_j > x_i\}\| + 1$, the final position of x_j, for $1 \le j \le n$. $\mathrm{Osc}(X)$ can thus be calculated by summing the "distances" that consecutive elements in X are apart in the sorted permutation of X. Hence, if we store the original positions of the elements when sorting, we can compute $\mathrm{Osc}(X)$ by a linear time scan afterwards. We leave as an open problem to compute or even approximate $\mathrm{Osc}(X)$ in $o(n \log(\mathrm{Osc}(X)/n))$ time.

Acknowledgments

We thank Svante Carlsson for stimulating discussions and careful reading of the manuscript. The observation that the algorithm by Gabow, Bentley, and Tarjan for constructing the Cartesian tree performs at most $2n - 3$ comparisons is due to Professor W. Kosters, Leiden State University, The Netherlands.

References

1. W. H. Burge, Sorting, trees, and measures of order, *Inform. and Control* **1** (1958), 181–197.
2. S. Carlsson, C. Levcopoulos, and O. Petersson, Sublinear merging and Natural Mergesort, *Algorithmica*, to appear; Preliminary version in "Proc. Internat. Symp. on Algorithms," Lect. Notes in Comput. Sci., Vol. 450, pp. 251–260, Springer-Verlag, New York/Berlin, 1990.
3. C. R. Cook and D. J. Kim, Best sorting algorithms for nearly sorted lists. *Comm. ACM* **23** (1980), 620–624.
4. E. W. Dijkstra, Smoothsort, an alternative to sorting in situ, *Sci. Comput. Programming* **1** (1982), 223–233.
5. V. Estivill-Castro and D. Wood, A new measure of presortedness, *Inform. Comput.* **83** (1989), 111–119.
6. H. N. Gabow, J. L. Bentley, and R. E. Tarjan, Scaling and related techniques for computational geometry, *in* "Proceedings, 16th Annu. ACM Symp. on Theory of Computing, 1984," pp. 135–143.
7. L. J. Guibas, E. M. McCreight, M. F. Plass, and J. R. Roberts, A new representation of linear lists, *in* "Proceedings, 9th Annu. ACM Symp. on Theory of Computing, 1977," pp. 49–60.
8. D. E. Knuth, "The Art of Computer Programming, Vol. 3: Sorting and Searching," Addison-Wesley, Reading, MA, 1973.

9. C. LEVCOPOULOS AND O. PETERSSON, Sorting shuffled monotone sequences, *Inform. and Comput.*, to appear; preliminary version in "Proceedings, 2nd Scandinavian Workshop on Algorithm Theory," Lect. Notes in Comput. Sci., Vol. 447, pp. 181–191, Springer-Verlag, New York/Berlin, 1990.
10. H. MANNILA, Measures of presortedness and optimal sorting algorithms, *IEEE Trans. Comput* **C-34** (1985), 318–325.
11. K. MEHLHORN, "Data Structures and Algorithms, Vol. 1: Sorting and Searching," Springer-Verlag, Berlin, 1984.
12. O. PETERSSON, "Adaptive Sorting," Ph.D. thesis, Department of Computer Science, Lund University, 1990.
13. O. PETERSSON AND A. M. MOFFAT, A Framework for Adaptive Sorting, *in* "Proceedings, 3rd Scandinavian Workshop on Algorithm Theory," Lect. Notes in Comput. Sci., Vol. 621, pp. 422–433, Springer-Verlag, New York/Berlin, 1992.
14. F. P. PREPARATA AND M. I. SHAMOS, "Computational Geometry: An Introduction," Springer-Verlag, Berlin, 1985.
15. J. VUILLEMIN, A unifying look at data structures, *Comm. ACM* **23** (1980), 229–237.